

Rivanna notebook pipeline — starter kit for Beau (bundle copy)

This file lives next to Beau’s scripts in `Ivy_Net/5-Manuscript/Beau_guide/`. Start with `README_beau.md` for a file manifest.

Maintainer: Charles (Ivy_Net / tenure pipeline).

Two layers: **this guide** explains the big picture and day-to-day steps; `pipe_job_beau.slurm` and `track_slurm_beau.sh` in **this folder** repeat the same ideas inline with `YOUR_*` **placeholders** you edit once.

Charles keeps his production launcher as `pipe_job.slurm` at repo root (minimal comments); an unchanged reference copy here is `pipe_job_reference_beau.slurm`. Beau’s annotated launcher is `pipe_job_beau.slurm` (default `NB_IN` → `540_tenure_pipeline_beau.ipynb` format example).

The `5-Manuscript/Beau_guide/` folder holds primary `*_beau.*` deliverables plus `*_reference_beau.*` copies of scripts the `540` notebook or repo mentions.

Sync hint: If narrative diverges, reconcile against `tenure/HPC_SLURM_PIPELINE_GUIDE.md` (canonical path outside this bundle).

Reading logs without assuming Linux fluency

Slurm writes plain-text **log files** next to the directory you were in when you typed `sbatch` (your **submit directory**). Charles recommends `cd` into the **Ivy_Net repo root** first so `*.out` / `*.err` land beside `tenure/` — same layout as `pipe_job.slurm`. You read logs with terminal commands — think “streaming Notepad,” where `tail -f` keeps showing new lines as Slurm appends them.

What is a JOBID?

When you run `sbatch 5-Manuscript/Beau_guide/pipe_job_beau.slurm` from repo root, Slurm replies with something like `Submitted batch job 11807601`. That number (**11807601**) is the **job ID**. It appears in log filenames so each run stays separate.

Two streams: `.out` vs `.err` (NOT “error-only”)

Unix separates normal printed output into two pipes:

Stream	Filename pattern	Typical contents
stdout	<code>slurm-<job_name>-<JOBID>.out</code>	<code>echo</code> lines from the batch script, many <code>print()</code> statements from Python.
stderr	<code>slurm-<job_name>-<JOBID>.err</code>	Progress bars (<code>tqdm</code>), warnings, status lines — lots of Jupyter stacks write here even when nothing failed .

So `.err` is often where you see motion first, not necessarily because anything broke.

What does `tail` do? What does `tail -f` do?

- `tail slurm-pipe_job_beau-11807601.err` shows only the **last ~10 lines** of that file — snapshot.
- `tail -f` (**follow**) keeps the terminal **attached**: whenever Slurm appends another line to that file on disk, your terminal prints it immediately — **near-live monitoring** while the compute node runs your notebook.

Stop watching with **Ctrl+C**. That **only stops tail**; it does **not** stop the Slurm job.

Running Charles’s helper script: `./5-Manuscript/Beau_guide/track_slurm_beau.sh`

Shell scripts must be marked **executable** once:

```
chmod +x 5-Manuscript/Beau_guide/track_slurm_beau.sh
```

What `chmod +x` means in plain English: “allow this file to run as a program.” Without it, `./5-Manuscript/Beau_guide/track_slurm_beau.sh` may say **Permission denied**.

The `./` means “run the script **from this path relative to where you** `cd` ‘d,’” not a system-wide installed command.

Run from **repo root**:

```
./5-Manuscript/Beau_guide/track_slurm_beau.sh 11807601
```

That finds `slurm-*11807601*.err` under the Ivy_Net root (or under `Beau_guide/` if that’s where logs landed) and `tail -f` it — same idea as typing `tail -f ..` yourself, but it picks the right filename.

1. DIY starter kit — what files to hand Beau

Give him enough to clone/run **the same workflow**, not necessarily every asset in Ivy_Net.

This folder (`5-Manuscript/Beau_guide/`) now includes: primary `*_beau.*` scripts, `README_beau.md` (manifest), `HPC_SLURM_PIPELINE_GUIDE_beau.md` (this file), `540_tenure_pipeline_beau.ipynb` (format example), `.env.example_beau`, and `*_reference_beau.*` copies of upstream Slurm/shell scripts the notebook or Charles mention. Use `README_beau.md` for a one-table file list.

Minimum (if he clones the whole repo)

What	Why
Full <code>Ivy_Net</code> repo	Notebook imports <code>functionsG_working.py</code> , <code>tenure/tenure_pipeline/*.py</code> , paths under <code>tenure/</code> .
Everything under <code>5-Manuscript/Beau_guide/</code>	Slurm/beau helpers + example notebook + reference scripts + <code>README_beau.md</code> .

`pipe_job_beau.slurm` defaults `NB_IN` to `5-Manuscript/Beau_guide/540_tenure_pipeline_beau.ipynb` (format example). For Charles’s production path, change `NB_IN` to `tenure/540_tenure_pipeline.ipynb`.

Nice additions

What	Why
<code>.env.example_beau</code> (copy to repo root <code>.env</code>)	Notebook loads <code>OPENALEX_*</code> etc. from repo-root <code>.env</code> .
Export list / <code>environment.yml</code>	So Beau can recreate <code>tenure_net</code> without guessing packages — coordinate with Charles .
Large data (DBLP XML, snapshots, CDH trees)	Only if Beau runs stages that read them — usually excluded from git; share separately or paths in Cell 0.

2. Overview — how the system fits together

Login node vs compute node

- **Login node**: fine for editing files and `sbatch`, bad for hour-long CPU/RAM/network jobs (fair-share policies).
- **Compute node**: Slurm reserves **CPUs**, **memory**, and **wall time** for one batch session.

One notebook, many partial runs

- **Cell 0** holds paths, constants, and `RUN_CELL*` booleans (one flag per stage).
- Later cells use `if RUN_CELL...:` to run work, or `else:` to skip and reload artifacts from disk when flags are `False`.
- Submitting `sbatch` runs the notebook **top to bottom once**, honoring those flags — no need to click “Run” in Jupyter.

Three artifacts every batch run produces

Files appear in the directory where `sbatch` was invoked — **recommended: Ivy_Net repo root** so logs sit next to `tenure/`:

File pattern	Contents
<code>slurm-<job_name>-<JOBID>.out</code>	Stdout — script echoes + much notebook <code>print</code> output. Use <code>tail -f</code> on this file to watch normal prints scroll by (see “Reading logs” above).
<code>slurm-<job_name>-<JOBID>.err</code>	Stderr — tqdm , warnings, much library chatter; often updates fastest. <code>track_slurm_beau.sh</code> opens this stream for you and <code>tail -f</code> follows it.
<code>slurm-<job_name>-<JOBID>-output.ipynb</code>	Full notebook copy with executed cell outputs — archive, proof-of-run, shareable artifact.

With `pipe_job_beau.slurm`, `job_name` defaults to `pipe_job_beau` — so `slurm-pipe_job_beau-11807601-output.ipynb` style names.

Execution engine

Inside the batch script: `papermill` (preferred; logs cell boundaries) or `jupyter nbconvert --execute` fallback. `PYTHONUNBUFFERED=1` + `stdbuf` push Python output out sooner so `tail -f` looks responsive instead of freezing until a huge buffer fills.

3. Step-by-step — Beau’s first Rivanna run

A. One-time setup

- 1. SSH to Rivanna (or use Open OnDemand terminal).
- 2. Clone Ivy_Net (or rsync Charles’s tree) into the path you want, e.g. `~/Ivy_Net` — remember this path; `sbatch` logs write next to where you submit.
- 3. Load conda module (same line as in `pipe_job_beau.slurm`):
`module load miniforge`
- 4. Create conda env whose name matches `ENV_NAME` in `pipe_job_beau.slurm` (default `tenure_net`):
`conda create -n tenure_net python=3.11`
Install `papermill`, `jupyter`, `pandas`, `tqdm`, and anything Cell 0 imports (`lxml`, `requests`, ...).
- 5. Copy secrets: if Charles uses OpenAlex or similar, put `OPENALEX_*` vars in repo-root `.env` (gitignored); match Charles’s pattern.

B. Edit Beau’s batch script (once per cluster/account)

Open `5-Manuscript/Beau_guide/pipe_job_beau.slurm` and replace every placeholder:

Marker	Put here
<code>YOUR_NETID@virginia.edu</code>	Your UVa email for Slurm notifications.
<code>YOUR_SLURM_ALLOCATION</code>	Your Rivanna billing group / PI allocation string (Research Computing docs or <code>accounts</code> command).
<code>## EDIT</code> sections	<code>--partition</code> , <code>--mem</code> , <code>--cpus-per-task</code> , <code>--time</code> if defaults don’t fit your notebook.
<code>NB_IN</code>	Default in <code>pipe_job_beau.slurm</code> is <code>5-Manuscript/Beau_guide/540_tenure_pipeline_beau.ipynb</code> . Use <code>tenure/540_tenure_pipeline.ipynb</code> for Charles production runs.
<code>ENV_NAME</code>	If you named the conda env something other than <code>tenure_net</code> , change the default or run <code>ENV_NAME=myenv sbatch 5-Manuscript/Beau_guide/pipe_job_beau.slurm</code> .

If conda env binaries live outside `~/conda/envs/`, edit the `CONDA_PYTHON` / `CONDA_PAPERMILL` / `CONDA_NBCONVERT` lines at the bottom of `5-Manuscript/Beau_guide/pipe_job_beau.slurm`.

C. Configure the notebook run

1. Open `5-Manuscript/Beau_guide/540_tenure_pipeline_beau.ipynb` (example), or `tenure/540_tenure_pipeline.ipynb` if `NB_IN` points there.
2. In **Cell 0**, set `RUN_CELL*` booleans for this submission only.
3. **Save** the notebook.

D. Submit and monitor

```
cd ~/Ivy_Net                                # INSERT: your actual Ivy_Net repo root path
sbatch 5-Manuscript/Beau_guide/pipe_job_beau.slurm
```

Note the numeric **JOBID** Slurm prints (see “Reading logs” if this is unfamiliar).

Stdout — watch `print`-heavy lines:

```
tail -f slurm-pipe_job_beau-<JOBID>.out
```

Stderr (often better “heartbeat”; tqdm lives here):

```
chmod +x 5-Manuscript/Beau_guide/track_slurm_beau.sh    # once per clone
./5-Manuscript/Beau_guide/track_slurm_beau.sh <JOBID>
```

Or omit JOBID to follow the newest `slurm-*.err` found under repo root or `Beau_guide/`.

Queue:

```
squeue -u $USER
watch -n 5 'squeue -u $USER'
```

E. After the job finishes

- Open `slurm-pipe_job_beau-<JOBID>-output.ipynb` in Jupyter or VS Code — full outputs embedded.
- If the job failed, read tail of `.err` first, then `.out`, then email from Slurm.

F. Cancel a running job

```
scancel <JOBID>
```

4. Naming note

Beau mentioned `pip_job_beau` — the checked-in Slurm script is `pipe_job_beau.slurm` (**pipe**, short for pipeline job, not Python `pip`) under `5-Manuscript/Beau_guide/`, so job names and `sbatch` paths stay readable.

5. Official Rivanna references

- Slurm overview / partitions / `--account`: [Research Computing — Rivanna user guide](#)
- `module load` stacks change rarely but verify `miniforge` name if yours fails.

6. Troubleshooting quick hits

Symptom	Check
<code>conda env not found</code>	<code>conda env list</code> — name matches <code>ENV_NAME</code> ; <code>module load miniforge</code> before <code>sbatch</code> is inside script already.
<code>Notebook ... not found</code>	<code>NB_IN</code> path relative to repo root; batch script <code>cd</code> s there automatically — ensure <code>functionsG_working.py</code> exists at Ivy_Net root; <code>sbatch</code> should be run from inside that tree's hierarchy so Slurm's submit dir resolves upward correctly.
<code>Invalid account</code>	<code>YOUR_SLURM_ALLOCATION</code> replaced with PI-approved string.
<code>Permission denied</code> running <code>./5-Manuscript/Beau_guide/track_slurm_beau.sh</code>	Run <code>chmod +x 5-Manuscript/Beau_guide/track_slurm_beau.sh</code> once (see "Reading logs").
Nothing new appears with <code>tail -f</code> ** for a long time**	Normal during slow imports or blocked I/O; eventually <code>PYTHONUNBUFFERED</code> + <code>stdbuf</code> flush lines — compare Beau script behavior with <code>pipe_job.slurm</code> if unsure.
<code>tail</code> wrong directory / missing <code>.err</code>	Logs land in <code>SLURM_SUBMIT_DIR</code> — whatever folder you <code>cd</code> 'd to before <code>sbatch</code> . Prefer <code>cd Ivy_Net</code> then <code>sbatch 5-Manuscript/Beau_guide/pipe_job_beau.slurm</code> ; <code>track_slurm_beau.sh</code> searches repo root and <code>Beau_guide/</code> .

Questions are mostly **allocation/partition** and **conda package parity** with **Charles** — everything else is already parameterized in `5-Manuscript/Beau_guide/pipe_job_beau.slurm`.